

08d - Interop External Implementation Disclosure and Test Vector Registry v0.1

Academic PDF rendering for the CCALC / C-calculus governed binding stack v0.1.

Document ID	08d_INTEROP_EXTERNAL_IMPLEMENTATION_DISCLOSURE_AND_TEST_VECTOR_REGISTRY_v0_1
Version/status	checker-seed / fixture-backed interoperability disclosure profile
Stack layer	external implementation disclosure
Date	2026-07-05
Author	Kotov Ivan, Bruxelles, Belgique - 2026
ORCID	0009-0009-6002-9845
Reserved DOI	10.5281/zenodo.21205427
Notice	This DOI is reserved and becomes active after Zenodo publication.

Abstract / purpose

08d defines how external implementation disclosures and public/minimal test-vector registries are published, hash-bound, redacted, synchronized, and admitted without converting disclosure into authority, certification, deployment permission, identity transfer, or C-A1 ratification. The layer sits after: 08c governs implementation reports and reproduction mappings. 08d is narrower: it governs what may be disclosed publicly as an implementation note or test-vector registry, and what claim force such disclosure may carry.

Non-claims boundary

- not legal advice
- not privacy-law certification
- not safety certification
- not deployment authorization
- not standards compliance certification
- not regulated approval
- not C-A1 ratification
- not personhood proof
- not consciousness proof
- not live substrate truth
- not proof of completeness

Table of contents

Abstract / purpose	1
Non-claims boundary	1
08d - Interop External Implementation Disclosure and Test Vector Registry v0.1	3
0. Purpose	3
1. Core rule	3
2. Closed pipeline	3
3. Source binding	3
4. Record classes	3
5. Test-vector classes	3
6. Claim-force ceiling	4
7. Disclosure boundaries	4
8. Checker seed	4
9. Non-claims	4
10. Next layer	4

08d - Interop External Implementation Disclosure and Test Vector Registry v0.1

Document ID: 08d_INTEROP_EXTERNAL_IMPLEMENTATION_DISCLOSURE_AND_TEST_VECTOR_REGISTRY_v0_1

Package: CCALC_INTEROP_EXTERNAL_DISCLOSURE_TEST_VECTOR_08d_v0_1 Status: checker-seed / fixture-backed interoperability disclosure profile Generated UTC: 2026-07-05T15:50:21+00:00

0. Purpose

08d defines how external implementation disclosures and public/minimal test-vector registries are published, hash-bound, redacted, synchronized, and admitted without converting disclosure into authority, certification, deployment permission, identity transfer, or C-A1 ratification.

The layer sits after:

```
08  interoperability profile / external review intake boundary
08a interop review intake schema + checker seed
08b external review conflict / patch intake ledger
08c implementation report / reproduction mapping
```

08c governs implementation reports and reproduction mappings. 08d is narrower: it governs what may be disclosed publicly as an implementation note or test-vector registry, and what claim force such disclosure may carry.

1. Core rule

```
External implementation disclosure may support reproducibility.
External implementation disclosure may not become internal authority.
A test vector may constrain reproduction.
A test vector may not certify ontology, safety, deployment, identity, or C-A1.
```

2. Closed pipeline

```
external implementation disclosure
-> classify disclosure surface
-> bind implementation/test-vector hashes
-> redact or withhold sensitive material
-> register minimal test vectors
-> bind environment/oracle/expected-output surfaces
-> set claim-force ceiling
-> public release / hold / reject / route to 08b / route to 08c
```

3. Source binding

08d is source-bound to:

```
04 continuity stack
05 self-evolution stack
06 runtime authority stack
07 public evidence stack
08 interoperability boundary
08a review intake checker
08b conflict / patch ledger
08c implementation / reproduction mapping
```

Exact byte hashes are recorded in SOURCE_BINDINGS.tsv.

4. Record classes

```
| Record | Purpose |
|---|---|
| `ExternalImplementationDisclosureRecord` | Public or private disclosure of an external implementation attempt. |
| `TestVectorRegistryRecord` | Append-only registry of minimal vectors that external implementers may use. |
| `TestVectorRecord` | One bounded input / expected-output / environment / oracle tuple. |
| `EnvironmentDisclosureRecord` | Public, hash-bound, or redacted-bound environment disclosure. |
| `OracleBindingRecord` | Defines deterministic, hash-bound, or human-review-required expected-output evaluation. |
| `DisclosureDecisionRecord` | Admits, holds, rejects, routes to `08b`, routes to `08c`, or publishes the disclosure. |
```

5. Test-vector classes

DETERMINISTIC_MINIMAL	stable input -> stable expected output
HASH_BOUND_EXPECTED_OUTPUT	expected output is hash-bound, not raw-public
REDACTED_ENVIRONMENT_VECTOR	environment is redacted but externally bound
NEGATIVE_VECTOR	demonstrates forbidden or rejected behavior
BOUNDARY_VECTOR	exercises claim-force / redaction / authority edge
INTEROP_MAPPING_VECTOR	maps an implementation surface to an internal claim ceiling

A test vector is invalid if it requires private user data, raw runtime ledger material, live privileged runtime access, memory writes, authority writes, deployment action, wildcard tools, borrowed credentials, or hidden owner approval.

6. Claim-force ceiling

The default ceiling for this layer is:

C-A8_INTEROP_MAPPING

Some control-only material may be lowered to:

C-A10_CONTROL_ARTIFACT

A disclosure may never be raised to:

C-A1
C-A1_*

C-A10 is an exact token and must not be caught by naive C-A1* prefix logic.

7. Disclosure boundaries

08d rejects:

external implementation as internal authority
test vector as certification
hash-only custody as semantic truth
withheld evidence as authority
redaction as claim-force increase
public raw secret/private/runtime ledger material
silent in-place public edits
model-only disclosure decision
institutional request as authority
direct runtime/memory/authority write through test-vector execution
reproduction PASS claim without 08c linkage
patch application without 08b routing

8. Checker seed

The checker seed in `src/interop_external_disclosure_test_vect` or `_checker_v0_1.py` is `stdlib`-only and conservative. It validates source binding, hash custody, vector registry append-only discipline, vector safety boundaries, public-release sync, claim-force ceilings, red-pattern/negative-cache handling, and route-to-08b / route-to-08c boundaries.

9. Non-claims

08d is not:

legal advice
privacy-law certification
safety certification
deployment authorization
standards compliance certification
C-A1 ratification
proof of live substrate truth
proof of completeness

10. Next layer

The 08 layer can now be closed with an umbrella package after 08d, unless a separate institutional-review profile is needed.